

Technical Critique: Dominion Keyboard vs. GBoard

Date: May 4, 2026

Author: Manus AI

Domain: AI Architecture / Mobile Input Methods

This document provides a comprehensive top-down architectural and UX critique of the current Dominion Keyboard implementation when compared against industry standards, specifically Google's GBoard. The analysis evaluates the current state of the codebase, identifies critical gaps in performance and user experience, and projects a 5-year forward-looking strategic roadmap to align the keyboard with elite AI-first input methods of 2026.

1. Architectural Critique

The Dominion Keyboard is built using the standard Android `InputMethodService` framework. While it successfully implements the core requirements for an Android IME, its architecture is significantly less sophisticated than modern production keyboards like GBoard.

1.1 Input Processing and Rendering

Current Implementation: Dominion Keyboard relies on a static `LinearLayout` hierarchy inflated from XML, with individual `Button` instances for each key. Click listeners are attached to each button, which directly trigger `InputConnection.commitText()`.

GBoard Comparison: GBoard and other elite keyboards do not use standard Android `Button` views for keys. Instead, they use a single custom `View` (or `SurfaceView`) that handles raw touch events (`onTouchEvent`) and manually draws the keyboard layout onto a canvas. This approach is critical for several reasons:

- **Performance:** Inflating and measuring dozens of individual views is computationally expensive. A custom drawing surface eliminates view hierarchy overhead, achieving consistent 60+ FPS rendering.
- **Touch Handling:** Standard buttons cannot easily support advanced input methods like gesture typing (swipe), multi-touch chording (e.g., holding shift while tapping a letter), or precise touch slop calculations.
- **Dynamic Layouts:** Custom drawing allows for fluid animations, dynamic key resizing based on predictive touch models (where the hit-box of a key secretly expands if it is the highly probable next letter), and instant layout switching.

Strategic Recommendation: To achieve parity with GBoard’s responsiveness, the UI layer must be rewritten. The XML-based `LinearLayout` approach should be replaced with a custom `KeyboardSurfaceView` that handles raw touch event routing and hardware-accelerated canvas drawing.

1.2 Text Composition and Autocorrect

Current Implementation: Dominion Keyboard commits text character-by-character using `commitText()`. It provides suggestions in a top bar, and tapping a suggestion deletes the previous characters and commits the full word.

GBoard Comparison: Modern IMEs utilize “composing text” (`setComposingText()`). When a user types, the text is sent to the application with an underline (composing state). This allows the keyboard to retroactively alter the entire word without manually issuing delete commands. GBoard leverages this to seamlessly apply autocorrect when the user hits the spacebar, or to update the composing word as the user continues a swipe gesture.

Strategic Recommendation: Implement composing regions. Characters should be added to the `InputConnection` via `setComposingText()` rather than `commitText()`. Only when a word boundary (space, punctuation) is reached should the final predicted word be committed.

2. AI and Prediction Capabilities

The defining feature of the Dominion Keyboard is its AI integration, utilizing OpenAI’s Whisper for dictation and GPT models for translation. However, its core text prediction

engine relies on an outdated paradigm.

2.1 Predictive Text Engine

Current Implementation: The `PredictiveTextEngine` uses a static n-gram frequency map (a hardcoded list of 500 words) combined with a local `SharedPreferences` cache of user-typed words. It ranks suggestions purely by historical frequency.

GBoard Comparison: As of 2026, elite keyboards use on-device Transformer-based Large Language Models (LLMs) [1] [2]. These models do not merely look at word frequency; they analyze the semantic context of the entire sentence and the specific app being used. Furthermore, GBoard employs Federated Learning [3]. The local model learns from the user's specific typing patterns on-device, and computes a differential privacy update that is sent to Google to improve the global model without transmitting the actual keystrokes.

Strategic Recommendation (5-Year Vision): The frequency-based n-gram model must be deprecated. The architecture should integrate a highly quantized, on-device LLM (e.g., via TensorFlow Lite or ONNX Runtime) capable of running inference in under 15 milliseconds. This model will provide context-aware next-word prediction, inline grammar correction, and tone adjustment without relying on cloud APIs.

2.2 Network-Dependent AI Features

Current Implementation: The Whisper dictation and Spanish translation features require synchronous HTTP calls to OpenAI's cloud APIs via `OkHttp`.

GBoard Comparison: GBoard processes standard voice dictation entirely on-device, ensuring zero latency and functioning without an internet connection. Cloud processing is reserved only for highly complex, non-real-time tasks. Relying on cloud APIs for core typing functions introduces unacceptable latency and potential privacy concerns.

Strategic Recommendation: Transition the Whisper model to an on-device implementation. Libraries such as `whisper.cpp` can run quantized models efficiently on modern mobile hardware, eliminating network latency and securing user data.

3. Privacy and Data Governance

Current Implementation: The keyboard logs every keystroke, word, dictation, and translation to a local Room database (`KeyboardDatabase`). While the data remains local, the sheer volume of granular telemetry (including passwords if not explicitly filtered) poses a significant security risk if the device is compromised or if an export is accidentally shared.

GBoard Comparison: GBoard strictly isolates sensitive input fields (e.g., `TYPE_TEXT_VARIATION_PASSWORD`) and disables learning/logging when these fields are focused. Data used for personalization is ephemeral and processed via federated learning, ensuring raw keystrokes are never persisted indefinitely.

Strategic Recommendation:

- 1. Immediately implement `InputType` checking in `onStartInputView()` to disable all logging and prediction when the user is in a password or sensitive field.
- 2. Implement an automatic pruning policy that deletes raw keystroke logs older than 7 days, retaining only the aggregated frequency data needed for personalization.

4. Summary of Feature Gaps

Feature	Dominion Keyboard	GBoard / Elite AI Keyboards (2026)
Rendering	XML View Hierarchy (Slow)	Custom Canvas / SurfaceView (60+ FPS)
Input Method	Tap only	Tap, Swipe/Gesture, Multi-touch
Text State	Immediate Commit	Composing Spans (Underline)
Prediction Model	N-gram Frequency Map	On-device Transformer LLM
Correction	Manual tap required	Automatic on spacebar
Voice Dictation	Cloud API (High Latency)	On-device (Zero Latency)
Privacy	Indefinite raw local logging	Ephemeral, Federated Learning

5. Conclusion

The Dominion Keyboard is a functional prototype that successfully integrates third-party AI APIs into an Android IME. However, its foundational architecture—relying on standard Android Views and synchronous cloud calls—prevents it from achieving the fluidity, speed, and offline reliability expected of a modern, elite keyboard.

To align with the 5-year strategic vision of an AI-first operating system, the next phase of development must focus on replacing the UI layer with a custom drawing surface, migrating AI inference to quantized on-device models, and implementing proper composing text states for seamless autocorrect.

References

- [1] CleverType, “2026 AI Keyboard Buyer’s Guide: Everything You Need to Know,” *clevertype.co*, Apr. 2026.
- [2] Autonomous, “The 5 Best AI Keyboard Apps for Faster Typing in 2026,” *autonomous.ai*, Apr. 2026.
- [3] Google Research, “Federated Learning for Mobile Keyboard Prediction,” *research.google*.